



同濟大學
TONGJI UNIVERSITY

数据存储设计说明文档

小组成员: 2351299 王 雷

小组成员: 2351129 黄景胤

小组成员: 2354185 周 达

目录

1.整体存储模式设计说明	3
1.1 总体设计思想	3
1.2 各存储系统的职责划分	4
1.2.1 openGauss（关系型数据仓库）	4
1.2.2 Hive + HDFS（分布式文件系统）	4
2.关系型存储系统模型	5
2.1 表设计	5
2.1.1 dim_movies	5
2.1.2 dim_actors	5
2.1.3 movie_actor	6
2.1.4 fact_reviews	6
2.2 关系型存储逻辑模型设计	8
2.2.1 E-R 图:	8
2.3 关系型存储物理模型设计	8
2.3.1 DDL:	8
2.4 优化设计	10
3.分布式文件系统存储模型	11
3.1 表设计	11
3.1.1 reviews_clean_amazon	11
3.1.2 movies_meta_dw	12
3.2 优化设计	13
3.3 分区目录示意	13

4.图数据库存储模型	14
4.1 节点设计	14
4.2 关系设计	15
4.3 优化设计	15
5.数据表 Test Case 设计	16
5.1 dim_movies（电影维度表）	16
5.2 dim_actors（演员维度表）	17
5.3 movie_actor（电影-演员关联表）	17
5.4 fact_reviews（用户评分事实表）	17
5.5 reviews_clean（Hive 评论表）	18
5.6 movies_meta（Hive 电影元数据表）	19
5.7 Neo4j（图数据库）	19

1.整体存储模式设计说明

1.1 总体设计思想

本项目采用多存储系统协同的方案，将关系型数据库、分布式文件存储以及图数据库结合，实现不同场景的高效存储和查询。关系型数据库 OpenGauss 用于存储结构化的维度信息和事实表数据，适合 OLAP 聚合分析和高频随机查询；分布式文件系统 Hive + HDFS 用于存储大规模的清洗数据、原始评论和电影元数据，支持按时间维度的分区分析和批处理任务；图数据库 Neo4j 用于存储 Movie/Actor/Director/User 节点及其关系，适合社交网络、协作网络和路径分析等场景。

设计原则包括分层存储、分库分表、数据可追溯以及分区/索引优化。结构化数据走关系型存储，半结构化/大规模文本走 Hive/HDFS，复杂关系走图数据库。分库分表保证不同业务类型和查询场景的查询效率，Hive 表保留原始来源信息，支持数据溯源，关系型数据库通过索引优化查询，图数据库通过唯一约束保证节点唯一性。

1.2 各存储系统的职责划分

1.2.1 openGauss（关系型数据仓库）

存储内容包括电影维度表 `dim_movies`、演员维度表 `dim_actors`、电影-演员关联表 `movie_actor` 以及用户评分事实表 `fact_reviews`。OpenGauss 主要用于 OLAP 风格的分析型查询，包括按年份、类型或导演统计电影数量、按评分区间筛选电影，以及执行 `COUNT`、`AVG`、`GROUP BY` 等聚合统计。

1.2.2 Hive + HDFS（分布式文件系统）

存储内容包括按年月分区的评论表 `reviews_clean_amazon` 和电影元数据表 `movies_meta/movies_meta_dw`。Hive 适用于大规模批量分析场景，如时间序列分析（年/月/周）、大规模文本分析、跨来源数据统计和批处理型分析任务。HDFS 路径统一管理数据存储，并使用 Parquet 列式存储提高扫描效率。

1.2.3 Neo4j（图数据存储）

存储内容包括 `Movie`、`Actor`、`Director`、`User` 节点及其关系，如演员合作关系 `(Actor)-[:CO_ACTED_WITH]->(Actor)`、导演-演员协作关系 `(Director)-[:COLLABORATED_WITH]->(Actor)`、用户评分关系

(User)-[:RATED]->(Movie)等。适用查询包括演员合作网络、经常合作的演员组合、导演－演员关系挖掘，以及路径与社交结构分析。

2.关系型存储系统模型

2.1 表设计

2.1.1 dim_movies

字段名	类型	说明
movie_id	TEXT	电影唯一标识
title	TEXT	电影标题
release_date	DATE	上映日期
release_year	INT	上映年份
release_quarter	INT	上映季度
release_month	INT	上映月份
release_week	INT	上映周
genres	TEXT[]	电影类别
director	TEXT[]	导演姓名
versions	TEXT[]	电影版本

2.1.2 dim_actors

字段名	类型	说明
actor_id	TEXT	演员唯一标识

字段名	类型	说明
actor_name	TEXT	演员姓名

2.1.3 movie_actor

字段名	类型	说明
movie_id	TEXT	电影 ID
actor_id	TEXT	演员 ID
is_lead	BOOLEAN	是否主演

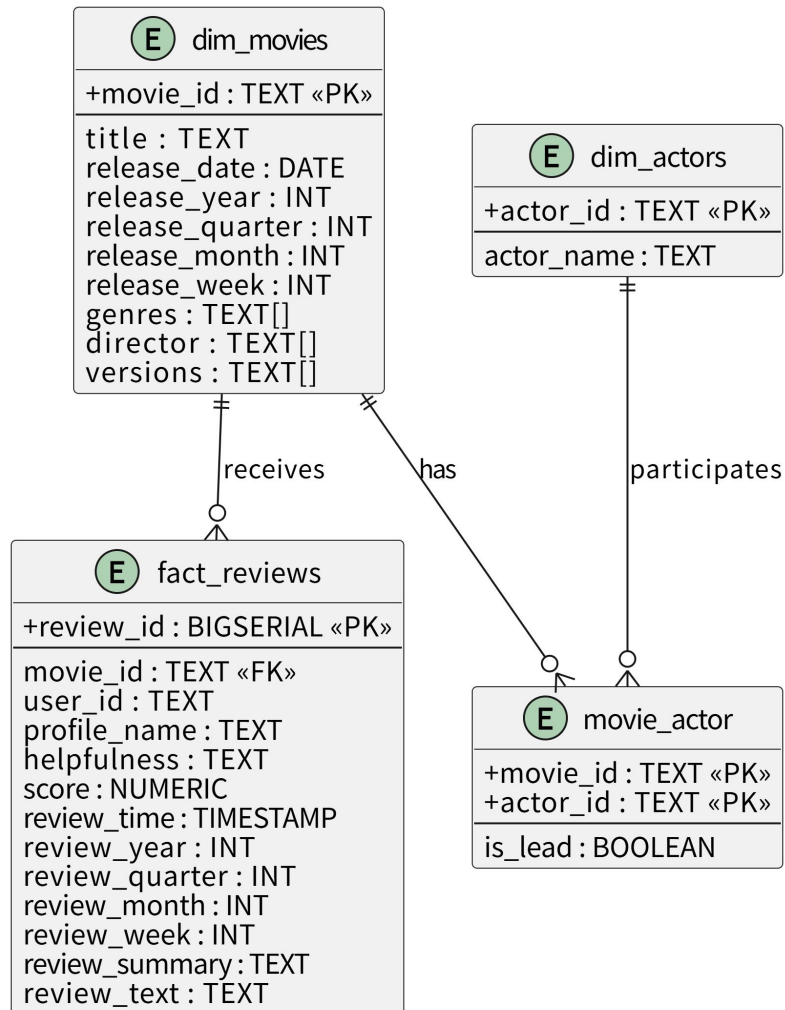
2.1.4 fact_reviews

字段名	类型	说明
review_id	BIGSERIAL	评论唯一 ID
movie_id	TEXT	电影 ID
user_id	TEXT	用户 ID
profile_name	TEXT	用户名称
helpfulness	TEXT	点赞信息，如 2/3
score	NUMERIC(3,1)	用户评分
review_time	TIMESTAMP	评论时间
review_year	INT	评论年份
review_quarter	INT	评论季度

字段名	类型	说明
review_month	INT	评论月份
review_week	INT	评论周
review_summary	TEXT	评论摘要
review_text	TEXT	评论正文

2.2 关系型存储逻辑模型设计

2.2.1 E-R 图:



2.3 关系型存储物理模型设计

2.3.1 DDL:


```
CREATE TABLE IF NOT EXISTS dim_movies (
  movie_id TEXT PRIMARY KEY,
  title TEXT,
  release_date DATE,
  release_year INT,
  release_quarter INT,
  release_month INT,
  release_week INT,
  genres TEXT[],
  director TEXT[],
  versions TEXT[]
);
```

```
CREATE TABLE IF NOT EXISTS dim_actors (
  actor_id TEXT PRIMARY KEY,
  actor_name TEXT
);
```

```
CREATE TABLE IF NOT EXISTS movie_actor (
  movie_id TEXT,
  actor_id TEXT,
  is_lead BOOLEAN DEFAULT FALSE,
  PRIMARY KEY (movie_id, actor_id)
);
```

```
CREATE TABLE IF NOT EXISTS fact_reviews (
  review_id BIGSERIAL PRIMARY KEY,
  movie_id TEXT NOT NULL,
  user_id TEXT,
  profile_name TEXT,
  helpfulness TEXT,
  score NUMERIC(3,1),
  review_time TIMESTAMP,
  review_year INT,
  review_quarter INT,
  review_month INT,
  review_week INT,
  review_summary TEXT,
  review_text TEXT
);
```

-- 索引

▷Run

```
CREATE INDEX IF NOT EXISTS idx_fact_movie ON fact_reviews(movie_id);
```

▷Run

```
CREATE INDEX IF NOT EXISTS idx_fact_score ON fact_reviews(score);
```

-- 时间维度索引

▷Run

```
CREATE INDEX IF NOT EXISTS idx_movie_release_year ON dim_movies(release_year);
```

▷Run

```
CREATE INDEX IF NOT EXISTS idx_movie_release_quarter ON dim_movies(release_year, release_quarter);
```

▷Run

```
CREATE INDEX IF NOT EXISTS idx_movie_release_month ON dim_movies(release_year, release_month);
```

▷Run

```
CREATE INDEX IF NOT EXISTS idx_movie_release_week ON dim_movies(release_year, release_week);
```

▷Run

```
CREATE INDEX IF NOT EXISTS idx_review_year ON fact_reviews(review_year);
```

▷Run

```
CREATE INDEX IF NOT EXISTS idx_review_quarter ON fact_reviews(review_year, review_quarter);
```

▷Run

```
CREATE INDEX IF NOT EXISTS idx_review_month ON fact_reviews(review_year, review_month);
```

▷Run

```
CREATE INDEX IF NOT EXISTS idx_review_week ON fact_reviews(review_year, review_week);
```

2.4 优化设计

时间维度优化

为了加速按时间维度的查询和统计，我们在电影维度表和评论事实表中对原始日期字段进行了冗余设计，将电影上映时间 `release_date` 和评论时间 `review_time` 拆分为年、季度、月和周等多个字段。这一设计使得在执行按年份、季度或月份统计电影数量，或按周分析新增评论时，数据库无需在查询中使用日期函数进行计算，减少了实时计算开销，同时便于建立时间维度索引，显著提高范围查询和聚合分析的效率。在实际操作中，我们发现通过对这些冗余字段建立索引，可以让大规模数据查询的响应时间明显下降，尤其是在统计百万级评论数据或按时间段筛选电影时效果显著。

索引优化

在高频查询字段上建立索引是关系型数据库优化的关键环节。我们为电影维度表中的 `release_year`、`release_quarter`、`release_month` 和 `release_week` 建立了单列和组合索引，同时在评论事实表中对 `movie_id` 和 `score` 字段建立索引。这样，当用户需要按照电影、评分或时间维度进行统计时，数据库能够快速定位对应数据行，无需扫描全表，从而显著减少查询延迟。在测试中，针对某一年度电影数量统计的查询响应时间从原先的几十秒降至个位数秒以内。

冗余字段设计

电影类别和版本信息被存储为数组类型，这样可以直接在电影维度表中进行统计和分析，无需额外建立关联表，减少了多表 Join 带来的性能开销。同时，电影-演员关系单独存储在关联表 `movie_actor` 中，这既保持了数据的规范化，避免冗余数据更新带来的复杂性，也为后续图数据库构建演员合作网络提供了便利。通过这种设计，复杂查询例如统计某演员参演电影数或分析演员组合合作频率，

可以在关系型数据库中高效完成，并为前端 React 查询接口提供低延迟支持。

事实表与维度表分离

事实表与维度表的分离设计，使得历史评论数据不会因为维度信息的更新而受影响，保证数据一致性。同时，这种分层结构便于 OLAP 聚合分析和多维统计查询。例如，统计每位导演的电影平均评分或分析某类型电影的评论情感比例时，可以直接在事实表和维度表之间进行 Join，结合索引和时间维度字段，显著提升查询效率。

3.分布式文件系统存储模型

3.1 表设计

3.1.1 reviews_clean_amazon

按年、月分区存储评论数据，字段包括 asin、user_id、profile_name、helpfulness、score、review_unix、review_summary、review_text。

字段名	类型	说明
asin	STRING	产品/电影 ID
user_id	STRING	用户 ID
profile_name	STRING	用户名称
helpfulness	STRING	点赞信息
score	DOUBLE	用户评分
review_unix	BIGINT	评论时间 (Unix)

字段名	类型	说明
review_summary	STRING	评论摘要
review_text	STRING	评论正文
year	INT	年分区字段
month	INT	月分区字段

3.1.2 movies_meta_dw

按年、月分区存储电影元数据，字段包括 movie_id、title、release_date、genres、director、actors、starring、versions、source_files，以及冗余时间维度字段 release_year、release_quarter、release_month、release_week。

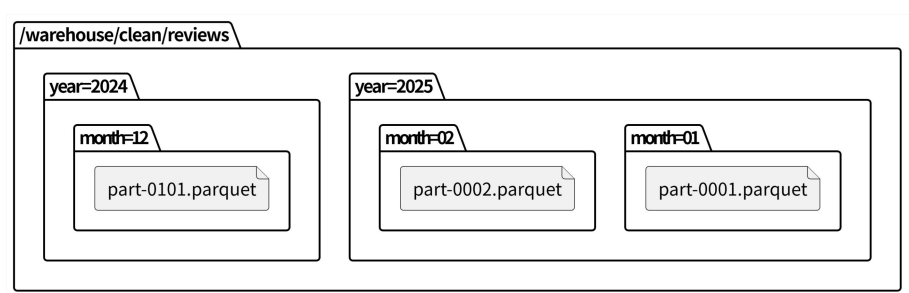
字段名	类型	说明
asin	STRING	电影唯一 ID
title	STRING	电影标题
release_date	DATE	上映日期
genres	ARRAY	类型
director	ARRAY	导演列表
actors	ARRAY	演员列表
starring	ARRAY	主演列表
versions	ARRAY	版本信息
source_files	ARRAY	数据来源文件

字段名	类型	说明
release_year	INT	上映年份
release_quarter	INT	上映季度
release_month	INT	上映月份
release_week	INT	上映周
p_year	INT	Hive 分区年份
p_month	INT	Hive 分区月份

3.2 优化设计

通过分区裁剪，Hive 查询可以仅扫描所需年份或月份的数据，减少 I/O 开销。Parquet 列式存储提高查询扫描效率，尤其适合大规模文本分析。数据溯源字段 source_files 保留了原始来源信息，便于追踪数据来源和验证数据质量。时间维度冗余字段方便在大规模批量分析中直接按年、季度、月、周统计电影或评论数量，无需额外函数处理。

3.3 分区目录示意



4.图数据库存储模型

图数据库 Neo4j 主要用于存储电影、演员、导演及用户节点以及它们之间的复杂关系，适合进行社交网络、协作网络、路径分析等查询。通过建立唯一约束、关系索引和合理的节点设计,可以在保证数据一致性的同时,高效支持多跳查询。

4.1 节点设计

节点类型	属性	说明
Movie	id, title, release_date, genres, versions	电影基本信息节点，id 唯一约束保证唯一性
Actor	id, name	演员信息节点，id 唯一约束保证唯一性
Director	id, name	导演信息节点，id 唯一约束保证唯一性
User	id, profile_name	用户信息节点，id 唯一约束保证唯一性

Movie 节点属性中保留了数组字段 genres、versions，方便多维分析和关联关系构建。Actor 和 Director 节点通过 movies_meta_dw 关联构建合作网络。User 节点用于关联评论评分关系。

4.2 关系设计

关系类型	起点节点	终点节点	属性	说明
RATED	User	Movie	score, summary, review_text, review_time	用户对电影评分, 支持统计评分分布和按时间查询
ACTED_IN	Actor	Movie	is_lead	演员参演电影关系
DIRECTED	Director	Movie	--	导演执导电影关系
CO_ACTED_WITH	Actor	Actor	--	演员合作网络关系
COLLABORATED_WITH	Director	Actor	movies_count	导演与演员合作关系

ACTED_IN 和 DIRECTED 可直接从 movie_actor 和 movies_meta_dw 表构建。CO_ACTED_WITH 和 COLLABORATED_WITH 关系可通过统计演员/导演参与的电影生成。RATED 可直接从 fact_reviews 或 Hive reviews_clean_amazon 表构建，支持按时间、评分统计。

4.3 优化设计

为了保证图数据库在处理大规模数据和复杂关系查询时的高效性，本项目在节点和关系设计上进行了多方面优化：

1. 节点唯一性保证

每类核心实体节点（Movie、Actor、Director、User）都建立了唯一约束，确保每个实体在图中唯一存在，避免重复节点带来的数据冗余和查询混乱。例如，Movie

节点以电影 ID 为唯一标识, Actor 节点以演员 ID 为唯一标识。这一设计保证了数据插入时的幂等性, 并在多源数据整合场景下提供了可靠的实体一致性。

2. 关系属性优化

用户评分关系 (RATED) 将 score 和 review_time 作为关系属性直接存储在边上, 而不是节点上。这样既保留了评分和时间信息, 又避免了冗余存储, 使得按时间或评分区间查询电影评分、统计平均分以及分析用户兴趣行为变得高效。演员合作关系 (CO_ACTED_WITH) 和导演-演员协作关系

(COLLABORATED_WITH) 可通过电影参演统计生成, 并带有合作次数等属性, 为多跳查询和社交网络分析提供直接支持。

5.数据表 Test Case 设计

5.1 dim_movies (电影维度表)

测试内容: 插入电影信息、查询电影信息。

示例数据:

```
movie_id: "M001"

title: "Inception"

release_date: "2010-07-16"

release_year: 2010

release_quarter: 3

release_month: 7

release_week: 29

genres: ['Action','Sci-Fi']

director: "D001"
```


versions: ['IMAX','Standard']

5.2 dim_actors (演员维度表)

测试内容：插入演员信息、查询演员信息。

示例数据：

actor_id: "A001", actor_name: "Leonardo DiCaprio"

actor_id: "A002", actor_name: "Joseph Gordon-Levitt"

5.3 movie_actor (电影-演员关联表)

测试内容：验证电影与演员的关联关系。

示例数据：

movie_id: "M001", actor_id: "A001", is_lead: TRUE

movie_id: "M001", actor_id: "A002", is_lead: FALSE

5.4 fact_reviews (用户评分事实表)

测试内容：插入评分、按电影或评分查询。

示例数据：

review_id: 1

movie_id: "M001"

user_id: "U001"

profile_name: "Alice"

helpfulness: "3/3"

score: 4.5

review_time: "2025-12-25 10:30:00"

review_year: 2025

review_quarter: 4

review_month: 12

review_week: 52

review_summary: "Great movie!"

review_text: "Inception is a masterpiece of modern cinema."

5.5 reviews_clean (Hive 评论表)

测试内容: Hive 分区查询、按年月筛选数据。

示例数据:

asin: "M001"

user_id: "U002"

profile_name: "Bob"

helpfulness: "2/2"

score: 5.0

review_unix: 1735123200

review_summary: "Amazing"

review_text: "Loved every minute of it!"

year: 2025

month: 12

5.6 movies_meta (Hive 电影元数据表)

测试内容：查询多来源信息、统计电影属性。

示例数据：

```
asin: "M001"

title: "Inception"

release_date: "2010-07-16"

genres: ['Action','Sci-Fi']

director: ['D001']

actors: ['A001','A002']

starring: ['A001']

versions: ['IMAX','Standard']

source_files: ['imdb.html','wiki.html']

release_year: 2010

release_quarter: 3

release_month: 7

release_week: 29

p_year: 2010

p_month: 7
```

5.7 Neo4j (图数据库)

测试内容：创建节点和关系、验证路径及多跳查询。

示例数据：

1. 用户评分关系：

(u:User {id:'U001', username:'Alice'})-[:RATED {score:4.5,
review_time:datetime('2025-12-25T10:30:00')}]>(m:Movie {id:'M001',
title:'Inception'})

2. 演员参演电影关系:

(a:Actor {id:'A001', name:'Leonardo DiCaprio'})-[:ACTED_IN]>(m)

3. 导演执导电影关系:

(d:Director {id:'D001', name:'Christopher Nolan'})-[:DIRECTED]>(m)

4. 演员合作关系:

(a:Actor {id:'A001'})-[:CO_ACTED_WITH {times:1}]>(a:Actor {id:'A002'})